Esencial Examen U1 y U2

- Esencial Examen U1 y U2
  - **GLOSARIO**
    - **Unidad 1**
    - **Unidad 2**
  - **TABLA COMPARATIVA - TRADUCCIÓN | COMPILACIÓN**
  - **TABLA COMPARATIVA - LENGUAJES DE PROGRAMACIÓN**
  - **TABLA COMPARATIVA - IDEs**
  - **TABLA COMPARATIVA - HERRAMIENTAS CASE**

GLOSARIO

Unidad 1

- **Bytecode**: Código interpretable directamente por una máquina virtual, como la JVM en Java.
- **Portabilidad**: Capacidad de un software para ser ejecutado en diferentes sistemas sin necesidad de modificaciones.
- **Firmware**: Software embebido en hardware que controla hardware específico. La BIOS de un ordenador es un ejemplo. Se almacena en memoria ROM o flash.
- **Instrucción**: Conjunto de datos insertados en una secuencia estructurada para que el procesador los interprete y ejecute.
- **Refactorizar código**: Proceso de optimización de código sin cambiar la lógica del mismo.
- **TypeScript**: Lenguaje de programación de código abierto. Es un superconjunto tipado de JavaScript que permite la detección temprana de errores relacionados con el tipo, haciendo que el código sea más fiable y menos propenso a errores.
- **FrameWork**: Conjunto de herramientas y bibliotecas que proporcionan soporte y facilita la programación. Proporciona una base completa sobre la que se puede empezar a construir el código. A diferencia de las librerías, el FrameWork es un marco de trabajo que brinda un set de herramientas para desarrollar sistemas o aplicaciones. Por lo general contiene librerías, pero no siempre. Ejemplos de FrameWorks:
    - AngularJs
    - Django
    - ASP.NET
- **Ejecutable**: Archivo que contiene un programa listo para ser ejecutado por el sistema operativo.
- **Paradigma**: Manera o estilo de programación. Existen diferentes paradigmas de la programación: Funcionales, Imperativos, Lógicos, Orientados a objetos, etc.
- **Clase** Plantilla o modelo a partir del cual se crean objetos en la Programación Orientada a Objetos (POO). Define atributos y métodos comunes para los objetos que se derivan de ella.
- **POO**: Programación Orientada a Objetos. Paradigma de programación basado en el concepto de "objetos", los cuales contienen en forma de campo unos atributos o variables (características) y el código en forma de métodos o funciones (comportamientos). El modelo a partir del cual se instancian los diferentes objetos del mismo tipo se le denomina "clase".
- **Servicio web**: Software intermediario que permiteque nuestra aplicación se comunique con otras aplicaciones a través de la web, utilizando protocolos estándar como HTTP y formatos de datos como XML o JSON.
- **Lenguaje Máquina**: Es el lenguaje nativo del ordenador, compuesto por instrucciones en código binario (0s y 1s). Es difícil de entender y escribir para los humanos. Las instrucciones no son portables entre diferentes arquitecturas de hardware, ya que cada tipo de procesador tiene su propio conjunto de instrucciones. Nadie programa directamente en lenguaje máquina hoy día.
- **Lenguaje de Alto Nivel**: Lenguajes de programación que están más cerca del lenguaje humano y son más fáciles de entender y escribir, suelen tener librerías y funciones predeterminadas y en ocasiones ofrecen frameworks para que la programación sea más intuitiva y eficiente. Suelen trabajar con orientación a objetos (POO). Ejemplos: Java, Python, C++, JavaScript.
- **Lenguaje de Bajo Nivel**: Lenguajes de programación que están más cerca del lenguaje máquina y son más difíciles de entender y escribir para los humanos. Permiten un mayor control sobre el hardware y la memoria, pero requieren un conocimiento más profundo de la arquitectura del sistema. Ejemplos: Ensamblador, C.

Unidad 2

- **IDE (Integrated Development Environment)**: Entorno de desarrollo integrado que proporciona un conjunto de herramientas para facilitar la programación, como editor de código, compilador, depurador y otras utilidades.
- **Editor de texto/código fuente**: Sección del IDE donde se escribe el código fuente del programa. Ofrece funcionens propias de la edición como copiar, cortar, pegar o buscar. Además, ofrece funciones adiciones como resaltado/coloreado de sintaxis para diferenciar variables, cadenas de caracteres, instrucciones, bucles, etc. De esta forma, se facilita la lectura del código.
- **Herramientas CASE (Computer-Aided Software Engineering)**: Herramientas que facilitan el desarrollo de software mediante la detección de errores, el soporte a la depuración de programas y pruebas, y la automatización de la documentación de una aplicación.
- **Compilador/Intérprete**: Herramienta que traduce el código fuente escrito en un lenguaje de programación a código máquina o a un lenguaje intermedio que pueda ser ejecutado por la máquina. Dependiendo del lenguaje, se utilizará un compilador (traduce todo el código de una vez) o un intérprete (traduce y ejecuta línea por línea).
- **Depurador (Debugger)**: Herramienta que permite identificar y corregir errores en el código fuente. Permite ejecutar el programa paso a paso, inspeccionar variables, establecer puntos de interrupción (breakpoints) y analizar el flujo de ejecución. El depurador permite detener el programa en cualquier punto de ruptura.
- **Constructor de interfaces gráficas (GUI Builder)**: Herramienta que facilita la creación de interfaces gráficas de usuario mediante un entorno visual. Permite arrastrar y soltar componentes gráficos (botones, cuadros de texto, menús, etc.) para diseñar la interfaz sin necesidad de escribir código manualmente. Algunas IDEs incorporan estas herramientas con el plugin correspondiente.

TABLA COMPARATIVA - TRADUCCIÓN | COMPILACIÓN

ASPECTO	TRADUCCIÓN (INTERPRETACIÓN)	COMPILACIÓN
DEFINICIÓN	Traduce y ejecuta el código en tiempo real mediante un intérprete.	Traduce todo el código fuente a código máquina antes de su ejecución.
MOMENTO DE EJECUCIÓN	Durante la ejecución del programa (línea por línea).	Antes de la ejecución (todo el código de una vez).
GENERA ARCHIVO EJECUTABLE INDEPENDIENTE	NO	SÍ..
DEPENDENCIA DEL CÓDIGO FUENTE	DEPENDIENTE	INDEPENDIENTE
EJEMPLOS	Python, JavaScript.	C, C++, Go.
SUBFASES	NO (traducción en tiempo real).	SI → **Análisis léxico** (Se detectan operaciones, caracteres de puntuación, etc.) | **Análisis sintáctico/semántico** (Se comprueba que las instrucciones tienen sentido y se ajustan a las reglas del lenguaje)
HERRAMIENTA	Intérprete.	Compilador.
VENTAJAS	Fácil depuración, portabilidad, no requiere compilación previa.	Mayor rendimiento, ejecutable optimizado, independiente del entorno.
DESVENTAJAS	Menor rendimiento, requiere el intérprete instalado.	Requiere compilación previa, ejecutable específico para hardware/software.

TABLA COMPARATIVA - LENGUAJES DE PROGRAMACIÓN

LENGUAJE	TIPO	FRAMEWORKS	ÁMBITOS DE APLICACIÓN
C	Compilado	N/P	Desarrollo de sistemas
C++	Compilado	Qt	Desarrollo de aplicaciones y juegos.
C#	Compilado	ASP.NET	Desarrollo de aplicaciones para Windows, móviles, juegos y aplicaciones web con (ASP.NET)
Java	Compilado	Spring	Aplicaciones móviles y de escritorio
Swift	Compilado	SwiftUI	Desarrollo de aplicaciones IOS, macOS
Rust	Compilado	Rocket	Desarrollo de sistema y aplicaciones web.
Python	Interpretado	Django	Desarrollo web y aplicaciones móviles
JavaScript	Interpretado	AngularJs	Desarrollo web, para el front-end y back-end
Ruby	Interpretado	Rails	Desarrollo web.
PHP	Interpretado	Laravel	Desarrollo Web del lado del servidor (Back-end)

TABLA COMPARATIVA - IDEs

IDE	LENGUAJE DESARROLLO IDE	SO	Lenguajes soportados	LICENCIA	
XCode	Swift/Objective-C	macOS	Swift, Objective-C	Gratuito (suscripción para publicar)	Sí
Netbeans	Java	Multiplataforma	Java, Python, PHP, HTML5, C/C++, JavaScript, CSS	Código abierto (Apache)	Sí
Eclipse	Java	Multiplataforma	Java, C++, C#, JavaScript, Python, PHP, etc.	EPL (Eclipse Public License)	Sí
Visual Studio	C++, C#	Windows	C++, C#, Python, HTML5, JavaScript, etc.	Propietaria	Sí
IntelliJ IDEA	Java, Kotlin	Multiplataforma	Java, Kotlin	Community (gratuito), Ultimate (pago)	Sí
Visual Studio Code	TypeScript, JavaScript	Multiplataforma	Múltiples lenguajes	Gratuito	Sí

TABLA COMPARATIVA - HERRAMIENTAS CASE

Herramienta CASE	Función	IDEs que la soportan
Depurador/Debugger	Identificar y corregir errores en el código mediante ejecución paso a paso.	XCode, Netbeans, Eclipse, Visual Studio, IntelliJ IDEA, Visual Studio Code
Profiler	Analiza el rendimiento de la aplicación durante la ejecución.	Netbeans, Eclipse, Visual Studio, IntelliJ IDEA, Visual Studio Code
Generación automática de documentación	Genera documentación HTML a partir de comentarios en el código (ej. Javadoc en Java).	Netbeans, Eclipse, IntelliJ IDEA
Gestión de módulos	Instalar, desinstalar y gestionar plugins o módulos para extender funcionalidades.	XCode, Netbeans, Eclipse, Visual Studio, IntelliJ IDEA, Visual Studio Code